

CSE234 Week 14:

Inheritance

Inheritance allows classes to, for lack of a better term, inherit data and operations from other classes. Inheritance also allows for C++ to implement run-time polymorphism. Inheritance is weird in C++ compared to pure object-oriented languages like Java or C#.

Basic Inheritance

The basic syntax for C++ inheritance is to put a colon after the name of class doing the inheriting followed by the name of the class it inherits from.

```
class Base {};  
class Derived : Base {};
```

Unlike most other languages, C++ has inheritance visibility specifiers. Classes can have `public`, `protected`, or `private` inheritance. By default, a class applies the `private` specifier. A struct will apply the `public` specifier. Below is a table showing the relationship between the inheritance visibility specifiers and class visibility regions:

Visibility Region	Inheritance Specifier		
	private	protected	public
private	Inaccessible	Inaccessible	Inaccessible
protected	Inaccessible	Protected	Protected
public	Inaccessible	Protected	Public

From the table, we can tell that derived classes can never access the private members of their base class, no matter which inheritance specifier is used. A private inheritance specifier indicates that derived classes can never access any members of the base class. Protected inheritance prevents objects of the derived class from accessing base class members, but the public and protected members of the base class

are still available in the class. Public inheritance allows public members of the base class to be accessed by objects. Let's look at a concrete example:

```
class Base {
public:
    int x;
protected:
    int y;
private:
    int z;
};

class PublicDerived: public Base {
public:
    // legal, x is public in Base
    void printX() { std::cout << x << std::endl; }
    // legal, y is protected in Base
    void printY() { std::cout << y << std::endl; }
    // not legal, z is private in Base
    // void printZ() { std::cout << z << std::endl; }
};

class ProtectedDerived: protected Base {
public:
    // legal, x is public in Base
    void printX() { std::cout << x << std::endl; }
    // legal, y is protected in Base
    void printY() { std::cout << y << std::endl; }
    // not legal, z is private in Base
    // void printZ() { std::cout << z << std::endl; }
};

class PrivateDerived: private Base {
public:
    // not legal, private inheritance
    // void printX() { std::cout << x << std::endl; }
    // not legal, private inheritance
    // void printY() { std::cout << y << std::endl; }
    // not legal, z is private in base
    // void printZ() { std::cout << z << std::endl; }
};

int main() {
    Base base;
    // okay, x is public
    base.x = 0;

    // illegal, y is protected. protected members
    // are only visible in derived classes
    // base.y = 0;

    // illegal, z is private
    // base.z = 0;
}
```

```

    PublicDerived pub;
    // legal, x is public in Base and public inheritance was used
    pub.x = 0;

    // illegal, y is protected in Base
    // y can only be accessed inside the class
    // pub.y = 0;

    // illegal, z is private in Base
    // pub.z = 0

    ProtectedDerived prot;
    // illegal, protected inheritance blocks objects
    // from accessing public members of the base class
    // prot.x = 0;

    // illegal, y is protected in Base
    // prot.y = 0;

    // illegal, z is private in Base
    // prot.z = 0;

    PrivateDerived priv;
    // illegal, private inheritance makes all base class members private
    // priv.x = 0;
    // priv.y = 0;
    // priv.z = 0;

    Base *basePointer;
    // legal, basePointer can be a Base
    basePointer = new Base;
    delete basePointer;
    // legal, public inheritance of Base
    basePointer = new PublicDerived;
    delete basePointer;
    // illegal, protected inheritance prevents this
    // basePointer = new ProtectedDerived;
    // illegal, private inheritance prevents this
    // basePointer = new PrivateDerived;

    return 0;
}

```

Calling Base Class Constructors

Since private members of the base class are not visible in derived classes, the derived class must be able to call the base class's constructor if it wants to set those fields. Initializer lists are used call base class constructors. They can also be used to set the fields of the class. An initializer list starts with a colon after a parameter list of the constructor. It consists of a comma separated list of the class's fields and base class(es) which are passed values.

```

class Base {
public:
    // initializer list for setting field
    Base(int x) : data(x) {}
private:
    int data;
};

class Derived1: public Base {
public:
    // initializer list for calling base constructor
    Derived1(int x) : Base(x) {}
};

class Derived2: public Base {
public:
    // initializer list for calling base class constructor and set field
    Derived2(int x, int y) : Base(x), value(y) {}
private:
    int value;
};

```

Virtual, Override, and Final

When a base class has a function defined, the derived classes will all be able to use that function:

```

class Base {
public:
    void printMessage() const {
        std::cout << "Message\n";
    }
};

class Derived: public Base {}

int main() {
    Base base;
    base.printMessage();

    Derived derived;
    // derived objects can also use the base class functions
    derived.printMessage();
}

```

Derived classes can also override the definition of base class methods:

```

class Base {
public:
    void printMessage() const {
        std::cout << "Message in Base\n";
    }
}

```

```

};

class Derived: public Base {
public:
    void printMessage() const {
        std::cout << "Message in Derived\n";
    }
};

int main() {
    Base base;
    // prints: Message in Base
    base.printMessage();

    Derived derived;
    // prints: Message in Derived
    derived.printMessage();

    return 0;
}

```

This is all well and good, but one of the most important features of derived objects is that they can be used in place of Base class objects. This is known as runtime polymorphism, since the program decides what version of a function the object should use at runtime. Base class pointers and references can be instantiated as derived class members:

```

int main() {
    // create base class pointer, and instantiate it
    // to a derived class object
    Base *base = new Derived;
    // prints: Message in Base
    base->printMessage();
    delete base;

    // create a derived class pointer
    Derived *derived = new Derived;
    // prints: Message in Derived
    derived->printMessage();
    delete derived;

    return 0;
}

```

Despite being set to a derived class object, the pointer `base` will still call the base class method. Runtime polymorphism (in C++, C#, Java, at least) is handled by a virtual method table (VMT), a hidden member added to classes by the compiler. The VMT tracks which methods of a class are inherited, and which are potentially overridden. When an object reference calls a function, the VMT is consulted to determine which version of the function to call. However, in C++, to be added to

the VMT a function must be explicitly labeled as `virtual`. If the function is not virtual, it is not added to the VMT, causing the version from `Base` to be called, since the original type of the pointer is `Base*`, even though we used `new` to instantiate it to a `Derived` object. Since `derived` has the type `Derived*`, it still calls the version of `printMessage()` defined in `Derived`. If we make the `printMessage()` function virtual, `base->printMessage()` will call the derived class's implementation.

```
class Base {
public:
    void printMessage() const {
        std::cout << "Message in Base\n";
    }
};

class Derived: public Base {
public:
    void printMessage() const {
        std::cout << "Message in Derived\n";
    }
};

int main() {
    // create base class pointer, and instantiate it
    // to a derived class object
    Base *base = new Derived;
    // prints: Message in Derived
    base->printMessage();
    delete base;

    // create a derived class pointer
    Derived *derived = new Derived;
    // prints: Message in Derived
    derived->printMessage();
    delete derived;

    return 0;
}
```

The `override` specifier can be applied to derived class implementations of virtual methods. This causes the compiler to check that the base class has a virtual method with a matching signature. The `override` specifier is not required to override a virtual function in a derived class, but it does tell the compiler to ensure the method is a valid override.

```
class Base {
public:
    virtual void printMessage() const {
```

```

        std::cout << "Message in Base\n";
    }
};

class Derived: public Base {
public:
    void printMessage() const override {
        std::cout << "Message in Derived\n";
    }
};

class Derived2: public Base {
public:
    // this function will not compile: it is missing const
    // therefore, it does not match the signature in Base
    void printMessage() override {
        std::cout << "Message in Derived2\n";
    }
};

```

The `final` specifier is used to tell the compiler that no derived classes can override this function. `final` can also be used on classes to indicate that they cannot be inherited from. `final` can offer performance benefits when you know a class won't be inherited from, or a function will not be overridden, since it can disable VMT checks when it is known that there will never be a derived method to call.

```

class Base {
public:
    virtual void printMessage() const {
        std::cout << "Message in Base\n";
    }
};

class Derived: public Base {
public:
    void printMessage() const override final {
        std::cout << "Message in Derived\n";
    }
};

class DoublyDerived2: public Derived {
public:
    // this function will not compile: this function was marked
    // final in Derived. Any class inheriting from Derived cannot
    // implement printMessage()
    void printMessage() const override {
        std::cout << "Message in Derived2\n";
    }
};

```

Destructors

Generally, it is smart to always mark base class destructors as virtual. This ensures that base class pointers that reference derived class objects will call the correct destructor. Otherwise, memory leaks can occur.

```
class Base {  
public:  
    virtual ~Base() = default;  
}
```

Pure Virtual Methods and "Interfaces"

A pure virtual function is a function that has no implementation in its base class. This is done by declaring the function virtual and setting it equal to 0.

```
class Base {  
public:  
    virtual void pureVirtual() = 0;  
};
```

A class that contains a pure virtual method cannot be instantiated directly. For example:

```
Base base;
```

This is illegal because any class that contains a pure virtual method is marked as "abstract" by the compiler. Abstract classes cannot be directly instantiated. They can be used as parameter types (if pass-by-reference or pointer) or be used to create pointers that are set to concrete implementations. C++ does not natively have an `interface` category like Java and C#. In C++, an interface is colloquially an abstract class with no data members, all functions that are pure virtual, and no private members.

```
class Interface {  
public:  
    virtual ~Interface() = default;  
    void foo() = 0;  
    void bar() = 0;  
};
```

Multiple Inheritance

C++ supports multiple inheritance, which is when one derived class can

have multiple parents. Most languages do not support this, or only support multiple inheritance from interfaces. C++ has no such restrictions.

```
class Camera {
public:
    virtual void recordVideo() { /*does something*/ }
};

class VideoPlayer {
public:
    virtual void playVideo() { /*does something*/ }
};

class WebcamFeed: public VideoPlayer, public Camera {
public:
    // can override methods from both classes
    void recordVideo() override { /*does something*/ }
    void playVideo() override { /*does something*/ }
};
```

To call a base class method from a derived class, the name of the base class can be used to scope the function call:

```
class Camera {
public:
    virtual Recording& recordVideo() { /*does something*/ }
};

class VideoPlayer {
public:
    virtual void playVideo(Recording& r) { /*does something*/ }
};

class WebcamFeed: public VideoPlayer, public Camera {
public:
    void streamVideo() {
        VideoPlayer::playVideo(Camera::recordVideo());
    }
};
```

Example: Animals

```
// animal is an interface
class Animal {
public:
    virtual ~Animal() = default;
    virtual void makeSound() const = 0;
    virtual void move() const = 0;
};

class Bird: public Animal {
```

```

public:
    Bird() {}
    void makeSound() const override {
        std::cout << "Chirp\n";
    }
    void move() const override {
        std::cout << "Taking Flight\n";
    }
};

// Mammal is abstract since it doesn't implement makeSound or move
class Mammal: public Animal {
public:
    Mammal() {}
    // needs to be virtual so it can also be final
    virtual bool hasFur() final { return true; }
};

class Seal: public Mammal {
public:
    Seal() {}
    void makeSound() const override {
        std::cout << "Ar Ar Ar\n";
    }
    void move() const override {
        std::cout << "Swimming\n";
    }
};

class Cat: public Mammal {
public:
    Cat() {}
    void makeSound() const override {
        std::cout << "Meow\n";
    }
    void move() const override {
        std::cout << "Running on all fours\n";
    }
};

```

Example: Game Objects

```

class Entity {
public:
    virtual ~Entity() = default;
    Entity(int maxHp, int maxMana, int maxStamina) {
        health = maxHp;
        mana = maxMana;
        stamina = maxStamina;
    }

    virtual void takeDamage(int damage) {
        health -= damage;
    }
}

```

```

        virtual int calcDamage() = 0;
protected:
    int health;
    int mana;
    int stamina;
};

class Player final: public Entity {
public:
    enum State { BLOCKING, ATTACKING, HEALING, CHARGING, CASTING, NONE };

    Player(int maxHp, int maxMana, int maxStamina, std::string n) :
        Entity(maxHp, maxMana, maxStamina), name(n),
        state(NONE), crit(0.05), charged(false)
    {}

    void setActionState(State s) { state = s; }

    void takeDamage(int damage) override final {
        if (state == BLOCKING) damage /= 3;
        if (state == CHARGING) damage *= 1.25;
        Entity::takeDamage(damage);
    }

    int calcDamage() override final {
        int damage;
        // ...
        return damage;
    }
private:
    State state;
    std::string name;
    double crit;
    bool charged;
};

class Goblin final: public Entity {
public:
    Goblin(): Entity(15, 0, 20) {}

    // uses Entity::takeDamage

    int calcDamage() override final {
        stamina -= 5;
        return 5;
    }
};

class Warlock final: public Entity {
public:
    Warlock() : Entity(25, 50, 10) {}

    void takeDamage(int damage) override final {
        if (mana > 25) {
            mana -= 5;
            damage /= 2;
        }
    }
};

```

```

        Entity::takeDamage(damage);
    }

    int calcDamage() {
        if (mana <= 0) {
            return 0;
        }
        else {
            mana -= 10;
            return 15;
        }
    }
};

```

Object Slicing

A problem in C++ inheritance, especially for programmers who are used to Java or C#, is object slicing. In C++, parameters are pass-by-value by default. If a pointer or reference needs to be passed, it must be done explicitly. C# and Java always pass class objects by reference. When a derived class object is passed when a base class parameter is expected, the additional properties and methods will be "sliced" off the object.

```

class Animal {
public:
    Animal(std::string n) : name(n) {}
    virtual void makeSound() const {
        std::cout << "sound\n";
    }
    virtual std::string getType() const {
        return "Animal";
    }
private:
    std::string name;
};

class Cat: public Animal {
public:
    Cat(std::string n) : Animal(n) {}
    void makeSound() const override {
        std::cout << "meow\n";
    }
    void std::string getType() const override {
        return "Cat";
    }
}

void printAnimal(Animal animal) {
    std::cout << "Type: " << animal.getType() << "\n";
    animal.makeSound();
}

int main() {

```

```
Cat cat("Whiskers");  
printAnimal(cat);  
  
return 0;  
}
```

The output of this program would be:

```
Type: Animal  
sound
```

This something of a square peg in round hole problem. When passing an object by value, the object passed needs to fit into the memory space of the specified class. If the class specified is a base class and the object that is being passed is of a derived class, this means that the compiler will only copy the properties and methods of the base class from the derived object, and will disregard the VMT and additional methods, data, and implementations of the derived class. To rectify this problem, we must either pass a reference or pointer of the Base class instead:

```
// any of these would work  
  
// reference  
void printAnimal(Animal& animal) {  
    std::cout << "Type: " << animal.getType() << "\n";  
    animal.makeSound();  
}  
  
// const reference  
void printAnimal(const Animal& animal) {  
    std::cout << "Type: " << animal.getType() << "\n";  
    animal.makeSound();  
}  
  
// pointer  
void printAnimal(Animal *animal) {  
    std::cout << "Type: " << animal->getType() << "\n";  
    animal->makeSound();  
}
```

Conclusion

This week, we discussed inheritance in C++. Inheritance allows for classes to inherit the methods and data of other classes. We typically call the classes being inherited from base classes, super classes, or parent classes. The classes doing the inheriting are called derived classes, subclasses, or child classes. C++ inheritance is more complex than other languages. For one, C++ does not treat objects as references

by default, meaning that much of the runtime polymorphism that occurs automatically in Java or C# does not happen in C++ unless using references or pointers. Additionally, C++ adds another layer of complexity thanks to the inheritance visibility specifiers and multiple inheritance. Object slicing occurs when a base class parameter is not passed by reference or pointer, leading to the calls with derived class objects as the argument "slicing" the derived classes fields and methods off the object.